SRA - D650 Simulation Code Explanation

Source: d:\Dropbox\Public Documents\PythonScripts\SRA\SRA - D650_simulation.py

This document explains the purpose and structure of the Python module `SRA - D650_simulation.py`. The script is a D650-focused version of the Shoulder Recognition Algorithm, or SRA. It analyses an N-body stellar simulation along the bar major axis and attempts to identify paired left and right shoulder features in the surface-density profile.

Executive Summary

- The script loads the D650 stellar particle file, currently `D650_stars.npy`, from the configured data directory.
- It cuts particles near the bar major axis, bins stellar mass along x, and converts the binned mass into a log surface-density profile.
- The profile is normalized, cleaned of NaN/infinite values, smoothed with a second-order Butterworth filter, and differentiated.
- Candidate shoulders are detected from first-derivative extrema and bounded using minima/maxima in the radius of curvature.
- If a valid left/right pair is found, the script measures geometry, mass, velocity dispersion, angular momentum, excess mass, and shoulder strength.
- The returned value is a NumPy structured array. Optional side effects are plots, an animated GIF, and a saved `.npy` shoulders file.

Important Source Locations

Lines 1-25	Module-level description of the SRA concept and the intended algorithmic flow.
Lines 43-47	Hard-coded D650 paths and identifiers: `DATA_DIR`, `D650_stars.npy`, model `D`, timestep `650`, and temp output directory.
Lines 52-177	Small utility functions for nearest-value lookup, GIF creation, NaN/inf interpolation helpers, and percentile mask generation.
Lines 183-1592	The main `run_SRA(...)` routine, including data loading, profile construction, smoothing, shoulder recognition, measurements, plotting, output saving, and return.

Lines 1512-1576	Definition of the structured output dtype and field names.

Inputs And Dependencies

The module depends on NumPy, Matplotlib, SciPy, ImageIO, and three local modules: `plot_settings`, `model_params`, and `rotations`. The local modules provide plot styling, model metadata/mass handling, time conversion, display names, and 3D rotation support.

- `DATA_DIR` points to `D:\Dropbox\Public Documents\UCLAN\MSc Research\Data`.
- `SINGLE_STARS_FILE` is fixed to `D650_stars.npy`.
- `SINGLE_MODEL` is fixed to `D`.
- `SINGLE_TIMESTEP` is fixed to `650`.
- The stellar data array is expected to contain at least `x`, `y`, `z`, `vx`, `vy`, and `vz`; `model_params.append_mass_to_data` adds `m` if needed.

Helper Functions

`find_nearest`	Returns the array value closest to a supplied target. The SRA uses this when mapping analytic boundary positions back to binned profile centres.
`convert_pngs_to_animated_gif`	Reads saved PNG profile plots and combines them into a looping GIF animation using ImageIO.
`nan_helper` / `inf_helper`	Return masks and index helpers used to linearly interpolate invalid profile values before smoothing.
`generate_mask_arrays`	Builds percentile-based Boolean masks from values within the current data selection.
`generate_mask_arrays_from_limits`	Builds masks from precomputed percentile limits, intended for action-space slicing based on the full population.

Main Routine: `run_SRA`

`run_SRA(...)` is the working algorithm. Although it still accepts many parameters from a broader multi-model version, this file is configured around a single D650 stellar file and a single timestamp. Internally it sets `timestamp = ['650']`, so `t_start` and `t_end` do not currently drive a timestep range.

The most important practical parameters are `by_z_layers`, `override_Data`, `override_bar_extent`, `theta_z`, `theta_x`, `do_plots`, `slope_cutoff`, `y_cut`, `bin_size`, `Butterworth`, `test_extrema`, and `max_extrema`.

Algorithm Flow

1. Set analysis constants: y cut, default x limits, bin width, smoothing offset, rejection thresholds, and the x range relative to the bar radius.
2. Choose output folders and suffixes. If rotations are requested, the output names include the rotation angles.
3. Decide z slicing. With no slicing, one full-profile slice is used. With `Absolute`, fixed kpc layers are used. With `Relative`, layers are generated in multiples of the vertical scale height `hz`.
4. Load the D650 particle file, add mass if needed, and optionally rotate/project the model using `rotations.rotate_model`.
5. Determine the bar radial extent. In this D650-focused file, the normal path uses `override_bar_extent`; otherwise the bar extent is zero.
6. Cut the particles near the major axis using `|y| < y_cut` and either a bar-scaled x range or the default +/-12 kpc range.
7. Bin stellar mass along x with `scipy.stats.binned_statistic`, convert to log surface density, and normalize the profile using global density limits.
8. Interpolate NaN and infinite values, smooth the normalized profile with a second-order Butterworth filter, then compute first and second numerical derivatives.
9. Compute radius of curvature using `((1 + deriv**2)**1.5) / abs(deriv2)`.
10. Find candidate left and right shoulders from first-derivative extrema, then use radius-of-curvature extrema to define clavicle and shoulder boundaries.
11. Reject weak or implausible candidates that are too thin, too close to x=0, outside the bar, too steep, overlapping, or too noisy when extrema testing is enabled.
12. For accepted shoulders, compute mass, velocity, scale-height, angular-momentum, excess-mass, strength, and width measurements.
13. Append one row to the structured output array for every processed z/action slice, using NaN fields when no valid shoulders are found.
14. Optionally save plots, make an animated GIF, save the structured `.npy` output, and return the `shoulders_array`.

How The Shoulder Detection Works

The density profile is treated as a one-dimensional signal along the bar. A shoulder is not identified directly from raw particle counts. Instead, the script smooths the profile, examines the slope and curvature of that smoothed signal, and looks for paired left/right features inside the bar.

- On the left side, potential shoulders start from minima in the first derivative at `x < 0`.
- On the right side, potential shoulders start from maxima in the first derivative at `x > 0`.
- The nearest radius-of-curvature minima around each candidate define the clavicle region.
- The shoulder borders are then derived from nearby curvature features, with the inner border set to the inner clavicle border.
- Both sides must pass validation before the profile is treated as having a shoulder pair.

Measurements Captured

When a valid shoulder pair is detected, the module records both geometry and physical diagnostics. If no valid pair is found, most shoulder-specific measurements are stored as NaN, but central profile quantities such as peak density and total mass are still available where calculated.

- Positions: clavicle centres, shoulder inner/outer edges, clavicle inner/outer edges, and bar radial extents.
- Slopes: first-derivative values at clavicle centres and shoulder boundaries.
- Masses: total galaxy mass, bar-cut mass, clavicle mass, shoulder mass, central mass, and left/right splits.
- Kinematics: vertical and radial velocity dispersions, median vertical/radial velocities, angular momentum, and left/right regional versions.
- Vertical/radial structure: median absolute z, scale-height-like standard deviations, and radial spread.
- Excess: left/right/total excess mass and fractional shoulder strengths.
- Widths: shoulder widths and clavicle widths for both sides.

Output Files And Return Value

The function always returns a NumPy structured array called `shoulders_array`. If `generate_shoulders_file=True`, it also saves that array as a `.npy` file in the model folder. If plotting is enabled, it writes profile plots to the `Temp` folder. If animation is enabled, it combines those plots into a GIF and removes the temporary PNG files afterwards.

Important Caveats

- This file is configured for the single D650 stellar file. It is not currently a general timestep loop despite the `t_start` and `t_end` parameters.
- `slice_actions=True` immediately raises a `ValueError` because this version does not load an actions file.
- If `override_Data` is supplied, it must include a mass column `m`, and `override_bar_extent` must also be supplied.
- The normal bar-extent-file path appears disabled in this version: `bre` is set to an empty list whether or not `override_bar_extent` is present.
- Several calculations depend on exact bin-centre matching after using `find_nearest`; this works because the nearest value is returned from the same bin-centre array.
- The code contains many plotting and measurement responsibilities inside one function, so future maintenance would be easier if data loading, profile construction, detection, measurement, and plotting were separated.

Plain-English Summary

In plain terms, the module takes the D650 stellar simulation, compresses the particles near the bar into a one-dimensional density profile, smooths that profile, and searches for the distinct broad shelf-like features known as shoulders. When the shoulders are convincing, it measures where they are, how wide they are, how much excess stellar mass they contain, and how their kinematics compare with the surrounding bar regions.